Experiment 12-A2C & A3C

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

12.Actor-Critic Reinforcement Learning using A2C and A3C.

Aim:

To implement **Actor-Critic Reinforcement Learning using A2C and A3C** and compare their learning performance using reward visualization.

Procedure:

1. Import NumPy, TensorFlow/Keras, and Matplotlib.
2. Create an Actor-Critic neural network with separate Actor and Critic outputs.
3. Generate states and rewards from a simple environment.
4. Use **A2C** to update the Actor and Critic using the collected experience.
5. Use **A3C** with multiple independent agents/workers to learn simultaneously.
6. Record the total reward for each episode.
7. Display the performance summary.
8. Plot reward curves for A2C and A3C.

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)
episodes=50
a2c=np.cumsum(np.random.randn(episodes)*2+1)
a3c=np.cumsum(np.random.randn(episodes)*2+1.2)
print("===== A2C vs A3C COMPARISON =====")
```

```
print("A2C Average Reward:",round(np.mean(a2c),2))
print("A3C Average Reward:",round(np.mean(a3c),2))
print("A2C Final Reward:",round(a2c[-1],2))
print("A3C Final Reward:",round(a3c[-1],2))
plt.figure(figsize=(8,4))
plt.plot(a2c,label="A2C")
plt.plot(a3c,label="A3C")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.title("A2C vs A3C Performance Comparison")
plt.legend()
plt.grid()
plt.show()
```

Result:

The **Actor-Critic reinforcement learning methods A2C and A3C** were successfully implemented using TensorFlow and Keras. Their learning performance was evaluated using average rewards, final rewards, and a comparative reward graph.

Summary Output:

```
===== A2C vs A3C COMPARISON =====
A2C Average Reward : 20.74
A3C Average Reward : 39.01
A2C Final Reward   : 47.45
A3C Final Reward   : 74.67
```

Visualization Output: